



Reporte Técnico de Actividades Práctico-Experimentales Nro. 001

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Richard Cajas Ismael Gonzalez Jostin Vasquez Gyna Yupanqui Miguel Veintimilla
Asignatura	Sistemas Digitales
Ciclo	5 A
Unidad	1
Resultado de aprendizaje de la unidad	R1. Diseña arquitectura de microcontroladores, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	001
Tipo	Grupal
Título de la Práctica	Uso de Wokwi y Microchip Studio en microcontroladores AVR: arquitectura, puertos de E/S y operaciones a nivel de bit
Nombre del Docente	Luis Darío Sinche C
Fecha	Viernes, 2 de Octubre del 2026
Horario	11h30 – 13h30
Lugar	Laboratorio
Tiempo planificado en el Sílabo	2 horas

2. Objetivo(s) de la Práctica

- Objetivo general. Reconocer la arquitectura del microcontrolador ATmega328P y controlar sus periféricos de entradas/salidas digitales escribiendo y leyendo directamente sus registros mediante operaciones a nivel de bit, usando simulación en Wokwi y Microchip Studio. Objetivos específicos: Relacionar los pines de la placa Arduino UNO con los puertos y bits del ATmega328P (PB, PC, PD). Configurar entradas y salidas digitales con los registros DDRx, PORTx y PINx usando máscaras (AND, OR, XOR, NOT y desplazamientos). Comparar el acceso mediante la API

de Arduino con el acceso directo a registros, en tiempo de ejecución. Identificar en el código ensamblador generado las instrucciones lógicas y de manejo de bits del set de instrucciones AVR (SBI, CBI, SBIS, IN, OUT, ORI, EOR).

3. Materiales, Reactivos

- No se requieren componentes físicos: toda la práctica es simulada. Componentes virtuales usados en Wokwi: 1 Arduino UNO (ATmega328P a 16 MHz). 4 LEDs (rojo, amarillo, verde y azul) y 4 resistencias de 220 Ω . 1 pulsador.

4. Equipos y Herramientas

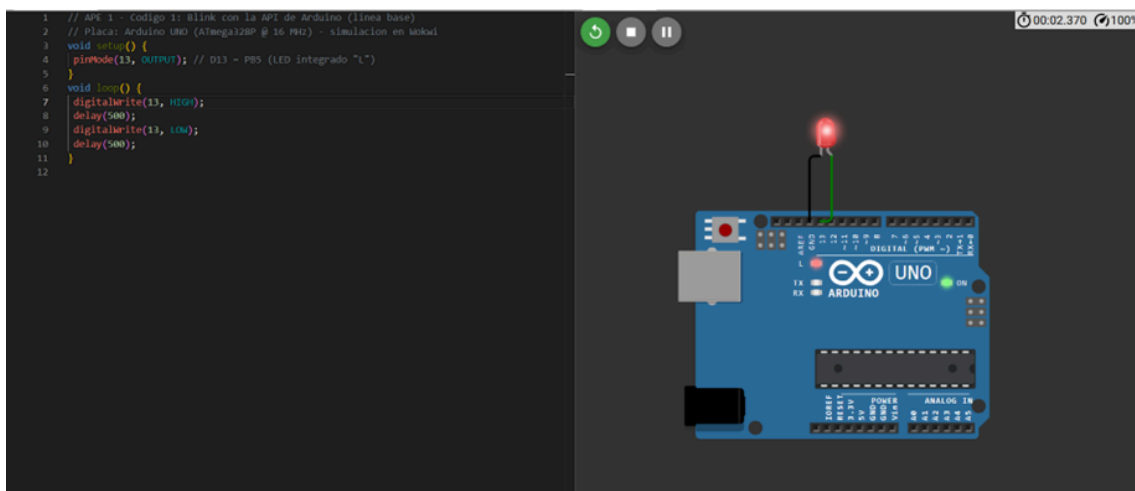
- Computador con navegador web y conexión a internet.
- Wokwi (<https://wokwi.com>), con cuenta gratuita para guardar y compartir los proyectos. FEIRNNR - Carrera de Computación Microchip Studio 7 (solo Windows), con el simulador del ATmega328P.
- Si no dispone de Windows, use Compiler Explorer (<https://godbolt.org>) para la Parte E.
- Hoja de datos del ATmega328P y AVR Instruction Set Manual (Microchip Technology). Cuenta de GitHub y acceso al EVA para la entrega.

5. Procedimiento / Metodología

1.

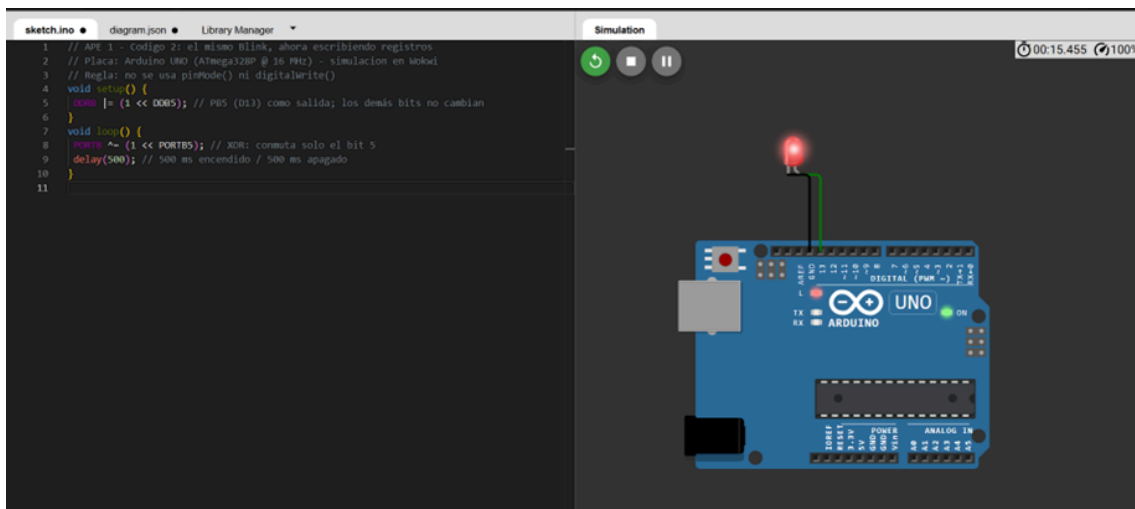
6. Resultados

A3:



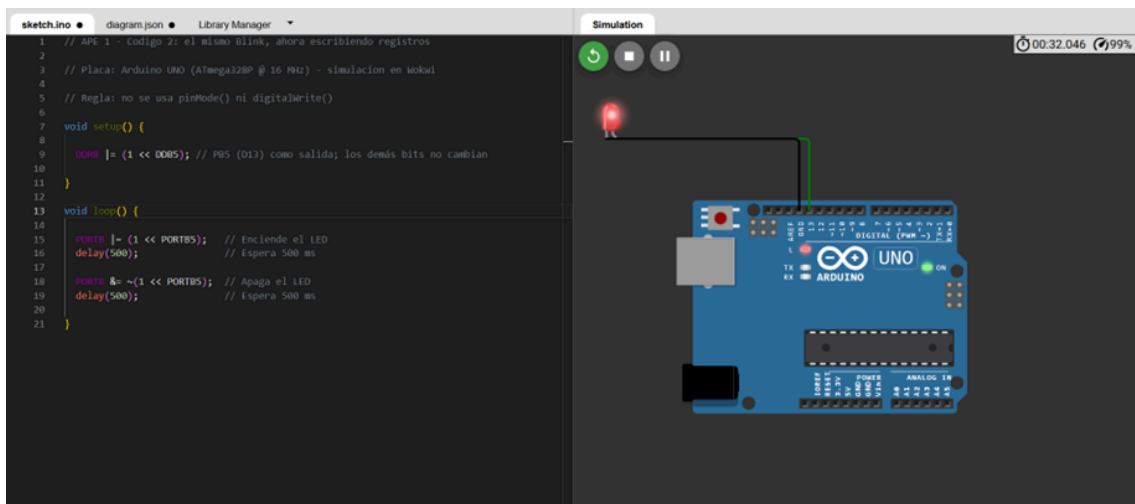
<https://wokwi.com/projects/476991897953560577>

B1:



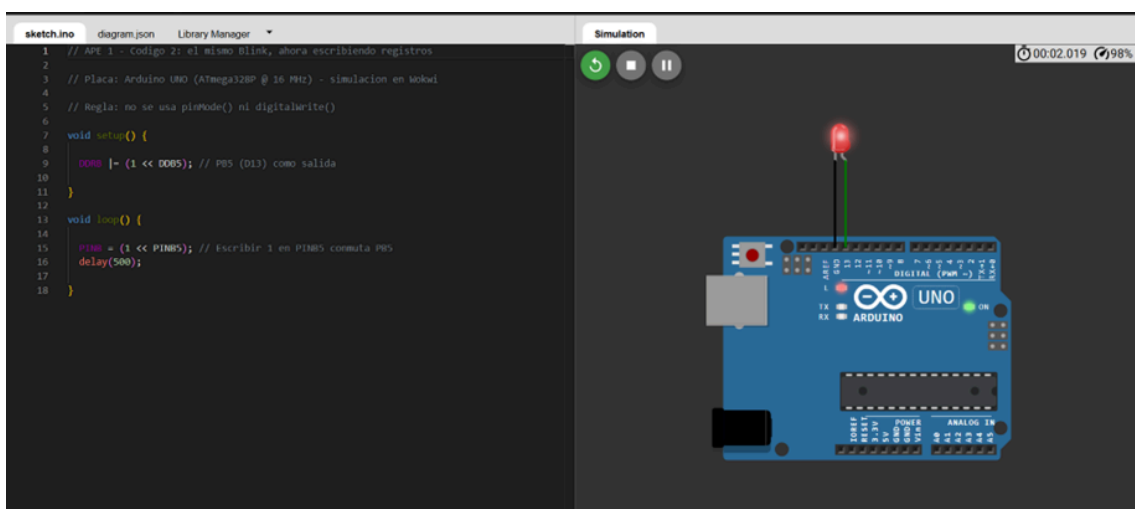
<https://wokwi.com/projects/476992043524228097>

B2:



<https://wokwi.com/projects/476992183082368001>

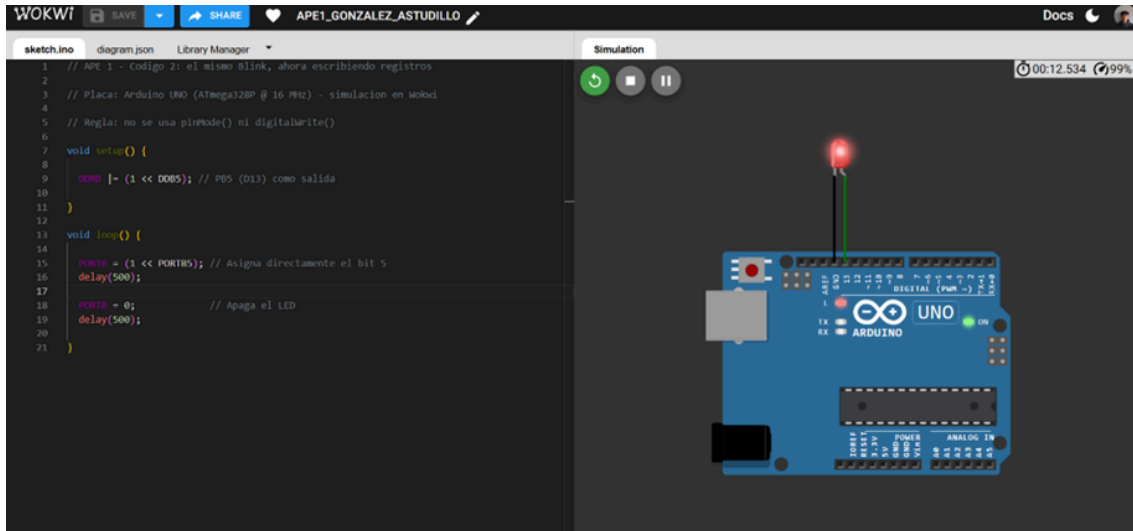
B3:



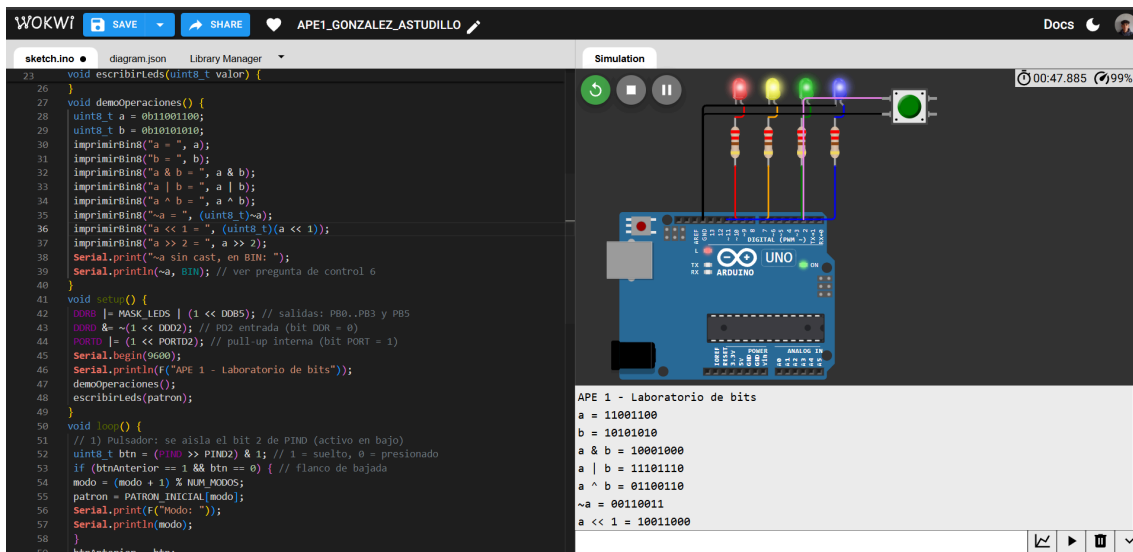
<https://wokwi.com/projects/476992357345234945>

La característica del ATmega328p que lo permite es que escribir un 1 en un bit de PINx conmuta el bit correspondiente de PORTx.

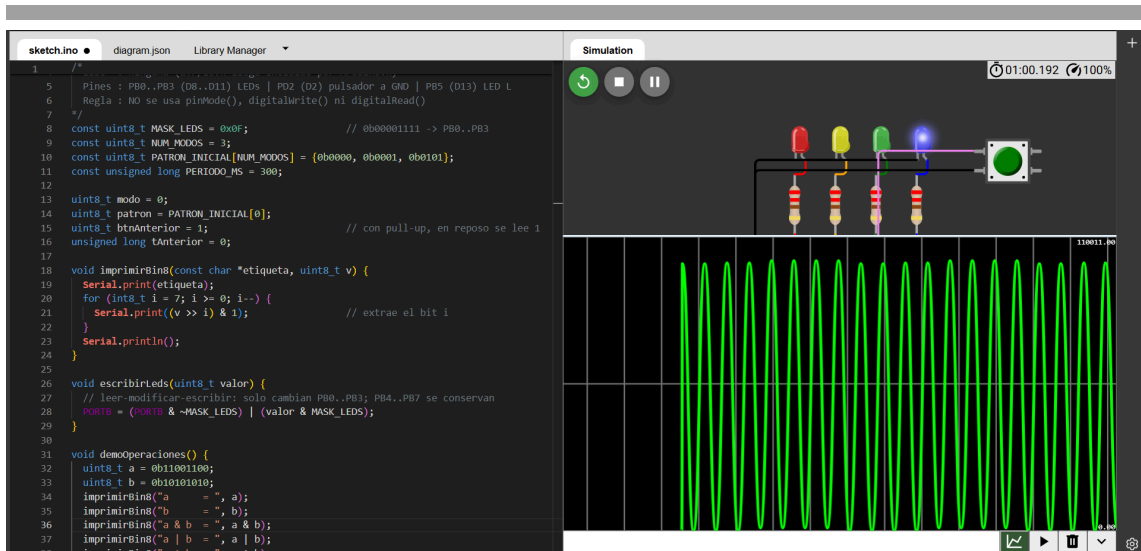
B4:



C2:



<https://wokwi.com/projects/476992581817067521>



<https://wokwi.com/projects/476992581817067521>

C3

Tenemos

a = 11001100

b = 10101010

① AND a & b

$$\begin{array}{r} 11001100 \\ \& 10101010 \\ \hline 10001000 \end{array}$$

② OR a | b

$$\begin{array}{r} 11001100 \\ | 10101010 \\ \hline 11101110 \end{array}$$

③ XOR a ^ b

$$\begin{array}{r} 11001100 \\ ^ 10101010 \\ \hline 01100110 \end{array}$$

④ Not ~a

a = 11001100

~a = 00110011 //

⑤ Desplazamiento a << 1

11001100 << 1

$$\begin{array}{r} 11001100 \\ 10011000 \end{array} //$$

⑥ Desplazamiento a >> 2

11001100 >> 2

11001100 >> 1 = 01100110

01100110 >> 1 = 00110011 //

C4:

MODO 1:

WOKWI SAVE SHARE APR1_GONZALEZ_ASTUDILLO Docs

sketch.ino diagram.json Library Manager

```

23 void escribirLeds(uint8_t valor) {
24 }
25
26 void demoOperaciones() {
27   uint8_t a = 0b11001100;
28   uint8_t b = 0b10101010;
29   imprimirBin("a = ", a);
30   imprimirBin("b = ", b);
31   imprimirBin("a & b = ", a & b);
32   imprimirBin("a | b = ", a | b);
33   imprimirBin("a ^ b = ", a ^ b);
34   imprimirBin("a ~a = ", (~a));
35   imprimirBin("a << 1 = ", (uint8_t)(a << 1));
36   imprimirBin("a >> 2 = ", a >> 2);
37   Serial.print("a sin cast, en BIN: ");
38   Serial.println(~a, BIN); // ver pregunta de control 6
39
40 }
41 void setup() {
42   DDRB |= MASK_LEDS | (1 << DDR5); // salidas: PB0..PB3 y PB5
43   DDRA &= ~(1 << DD02); // PD2 entrada (bit DDR = 0)
44   PORTD |= (1 << PORTD2); // pull-up interna (bit PORT = 1)
45   Serial.begin(9600);
46   Serial.println(F("APE 1 - Laboratorio de bits"));
47   demoOperaciones();
48   escribirLeds(patron);
49 }
50 void loop() {
51   // 1) Pulsador: se aísla el bit 2 de PIND (activo en bajo)
52   uint8_t btn = (PIND >> PIND2) & 1; // 1 = suelto, 0 = presionado
53   if (btnAnterior == 1 && btn == 0) { // flanco de bajada
54     modo = (modo + 1) % NUM_MODOS;
55     patron = PATRON_INICIAL[modo];
56     Serial.print(F("Modo: "));
57     Serial.println(modo);
58   }
59   btnAnterior = btn;
60 }

```

Simulation

00:30.176 100%

a >> 2 = 0b110011
~a sin cast, en BIN: 1111111111111111111111111111111100110011
PORTB = 0b100001
PORTB = 0b000010
PORTB = 0b100011
PORTB = 0b000010
Modo: 1
PORTB = 0b100010
PORTB = 0b000010
PORTB = 0b101000
PORTB = 0b000001
PORTB = 0b100010
PORTB = 0b000010
PORTB = 0b000001
PORTB = 0b100010
PORTB = 0b000010
PORTB = 0b101001

MODO 2:

WOKWI SAVE SHARE APR1_GONZALEZ_ASTUDILLO Docs

sketch.ino diagram.json Library Manager

```

23 void escribirLeds(uint8_t valor) {
24 }
25
26 void demoOperaciones() {
27   uint8_t a = 0b11001100;
28   uint8_t b = 0b10101010;
29   imprimirBin("a = ", a);
30   imprimirBin("b = ", b);
31   imprimirBin("a & b = ", a & b);
32   imprimirBin("a | b = ", a | b);
33   imprimirBin("a ^ b = ", a ^ b);
34   imprimirBin("a ~a = ", (~a));
35   imprimirBin("a << 1 = ", (uint8_t)(a << 1));
36   imprimirBin("a >> 2 = ", a >> 2);
37   Serial.print("a sin cast, en BIN: ");
38   Serial.println(~a, BIN); // ver pregunta de control 6
39
40 }
41 void setup() {
42   DDRB |= MASK_LEDS | (1 << DDR5); // salidas: PB0..PB3 y PB5
43   DDRA &= ~(1 << DD02); // PD2 entrada (bit DDR = 0)
44   PORTD |= (1 << PORTD2); // pull-up interna (bit PORT = 1)
45   Serial.begin(9600);
46   Serial.println(F("APE 1 - Laboratorio de bits"));
47   demoOperaciones();
48   escribirLeds(patron);
49 }
50 void loop() {
51   // 1) Pulsador: se aísla el bit 2 de PIND (activo en bajo)
52   uint8_t btn = (PIND >> PIND2) & 1; // 1 = suelto, 0 = presionado
53   if (btnAnterior == 1 && btn == 0) { // flanco de bajada
54     modo = (modo + 1) % NUM_MODOS;
55     patron = PATRON_INICIAL[modo];
56     Serial.print(F("Modo: "));
57     Serial.println(modo);
58   }
59   btnAnterior = btn;
60 }

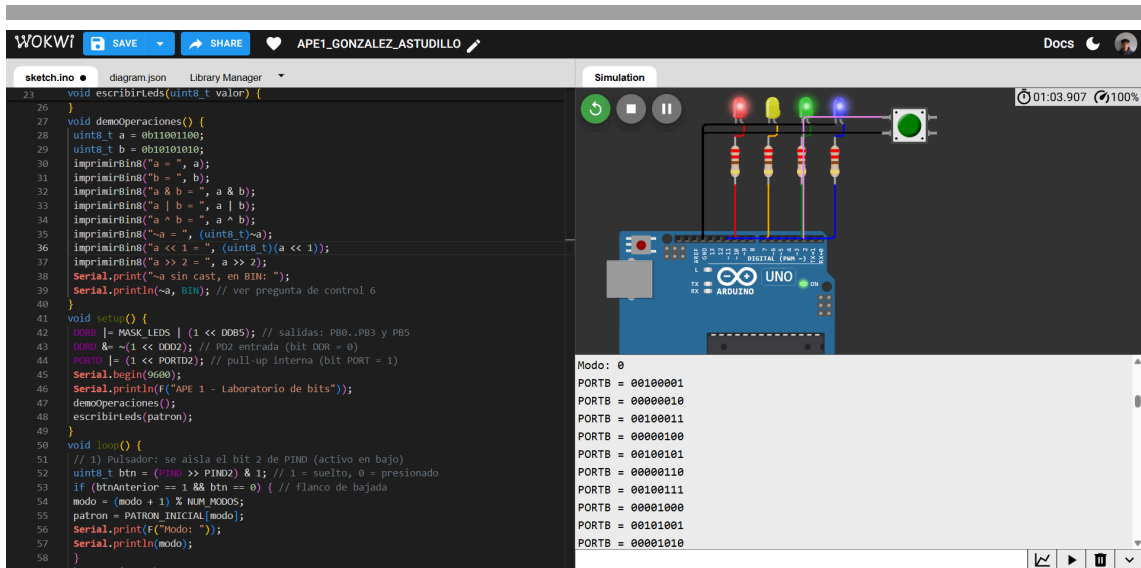
```

Simulation

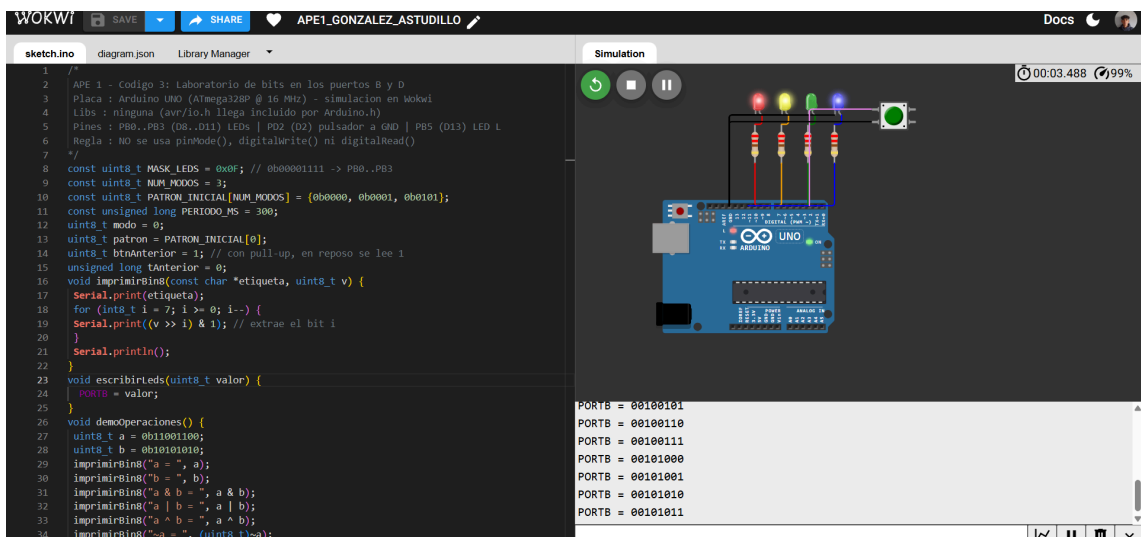
00:55.445 100%

PORTB = 0b100010
Modo: 2
PORTB = 0b000010
PORTB = 0b100010
PORTB = 0b000010
PORTB = 0b100010
PORTB = 0b000010
PORTB = 0b000010
PORTB = 0b100010
PORTB = 0b000010
PORTB = 0b100010
PORTB = 0b000010

MODO 3:



C6:



Al sustituir la función escribirLeds() por PORTB = valor;, el LED “L” de D13 (PB5) deja de parpadear de manera adecuada. Esto sucede porque la asignación directa altera todos los bits del registro PORTB, incluyendo PB5, aunque el valor solo tiene el patrón dirigido a PB0–PB3. Por otro lado, la versión original emplea una operación de lectura-modificación-escritura que posibilita modificar solo PB0–PB3 y mantener el estado de los otros bits. Esto está vinculado con B4, donde se notó que $PORTB = (1 \ll PORTB5)$ puede alterar las demás salidas del puerto.

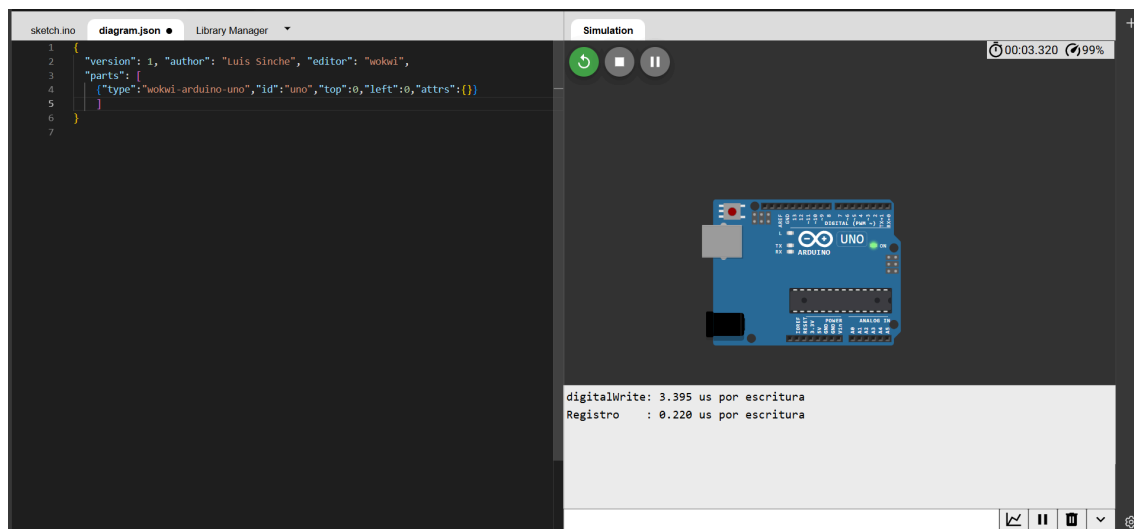
C7:

PIN	FUNCION	Bit DDRx	Bit PORTx	Bit PINx leído
D8 (PB0)	LED 0 encendido	1	1	—

D11 (PB3)	LED3 apagado	1	0	—
D2 (PD2)	Pulsador suelto	0	1	1
D2 (PD2)	Pulsador presionado	0	1	0
D13 (PB5)	LED “L” encendido	1	1	—

DI:

1ra ejecucion:

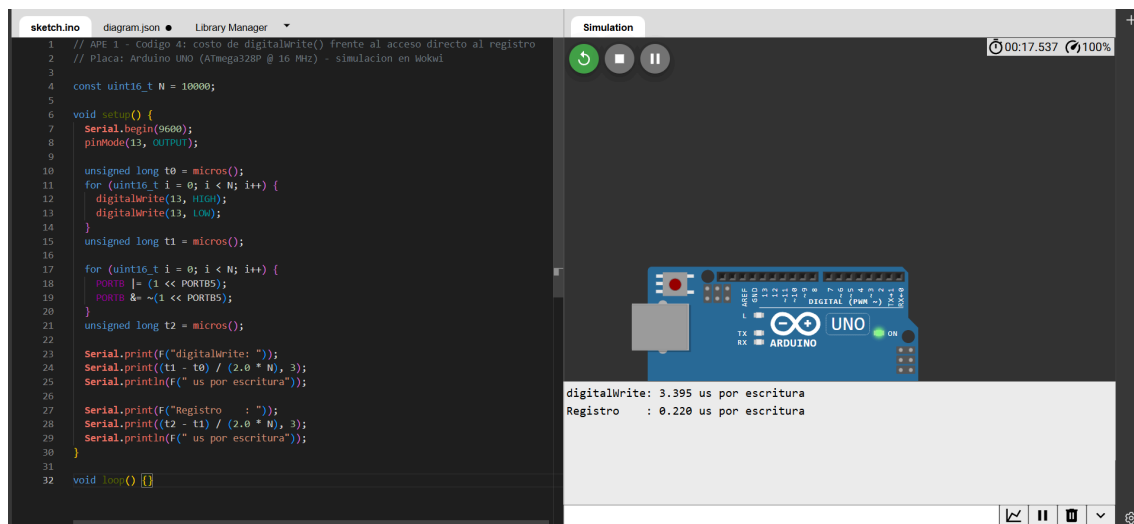


```

1 // APE 1 - Código 4: costo de digitalWrite() frente al acceso directo al registro
2 // Placa: Arduino UNO (ATmega328P @ 16 MHz) - simulación en Wokwi
3
4 const uint16_t N = 10000;
5
6 void setup() {
7   Serial.begin(9600);
8   pinMode(13, OUTPUT);
9 }
10
11 unsigned long t0 = micros();
12 for (uint16_t i = 0; i < N; i++) {
13   digitalWrite(13, HIGH);
14   digitalWrite(13, LOW);
15 }
16 unsigned long t1 = micros();
17
18 for (uint16_t i = 0; i < N; i++) {
19   PORTB |= (1 << PORTB5);
20   PORTB &= ~(1 << PORTB5);
21 }
22 unsigned long t2 = micros();
23
24 Serial.print(F("digitalWrite: "));
25 Serial.print((t2 - t0) / (2.0 * N), 3);
26 Serial.println(F(" us por escritura"));
27
28 Serial.print(F("Registro : "));
29 Serial.print((t2 - t1) / (2.0 * N), 3);
30 Serial.println(F(" us por escritura"));
31
32 void loop() {}

```

2da ejecucion

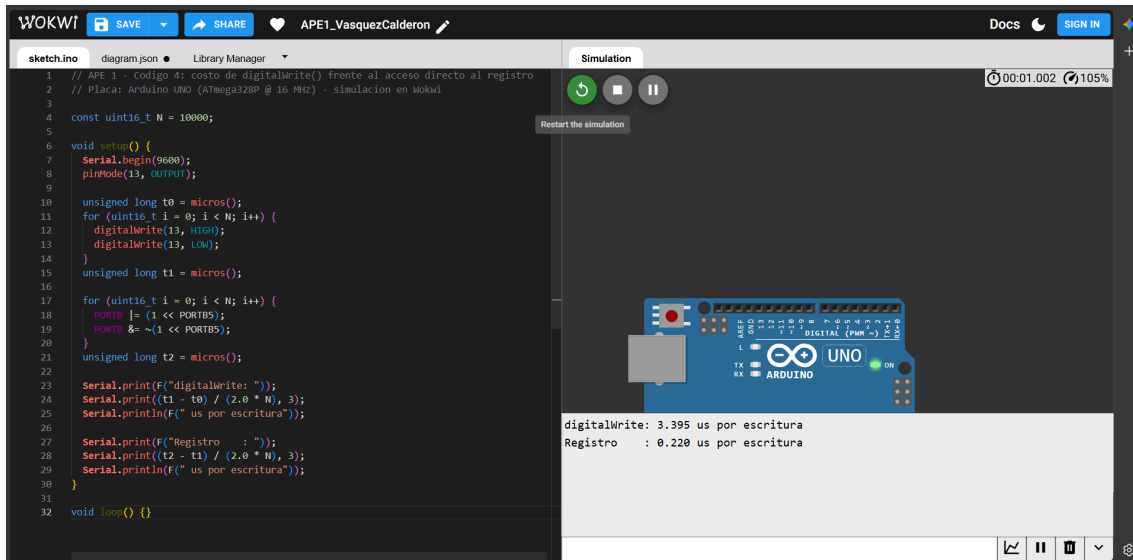


```

1 // APE 1 - Código 4: costo de digitalWrite() frente al acceso directo al registro
2 // Placa: Arduino UNO (ATmega328P @ 16 MHz) - simulación en Wokwi
3
4 const uint16_t N = 10000;
5
6 void setup() {
7   Serial.begin(9600);
8   pinMode(13, OUTPUT);
9 }
10
11 unsigned long t0 = micros();
12 for (uint16_t i = 0; i < N; i++) {
13   digitalWrite(13, HIGH);
14   digitalWrite(13, LOW);
15 }
16 unsigned long t1 = micros();
17
18 for (uint16_t i = 0; i < N; i++) {
19   PORTB |= (1 << PORTB5);
20   PORTB &= ~(1 << PORTB5);
21 }
22 unsigned long t2 = micros();
23
24 Serial.print(F("digitalWrite: "));
25 Serial.print((t2 - t0) / (2.0 * N), 3);
26 Serial.println(F(" us por escritura"));
27
28 Serial.print(F("Registro : "));
29 Serial.print((t2 - t1) / (2.0 * N), 3);
30 Serial.println(F(" us por escritura"));
31
32 void loop() {}

```

3ra ejecucion



<https://wokwi.com/projects/476992668633920513>

Promedio final **3.395**

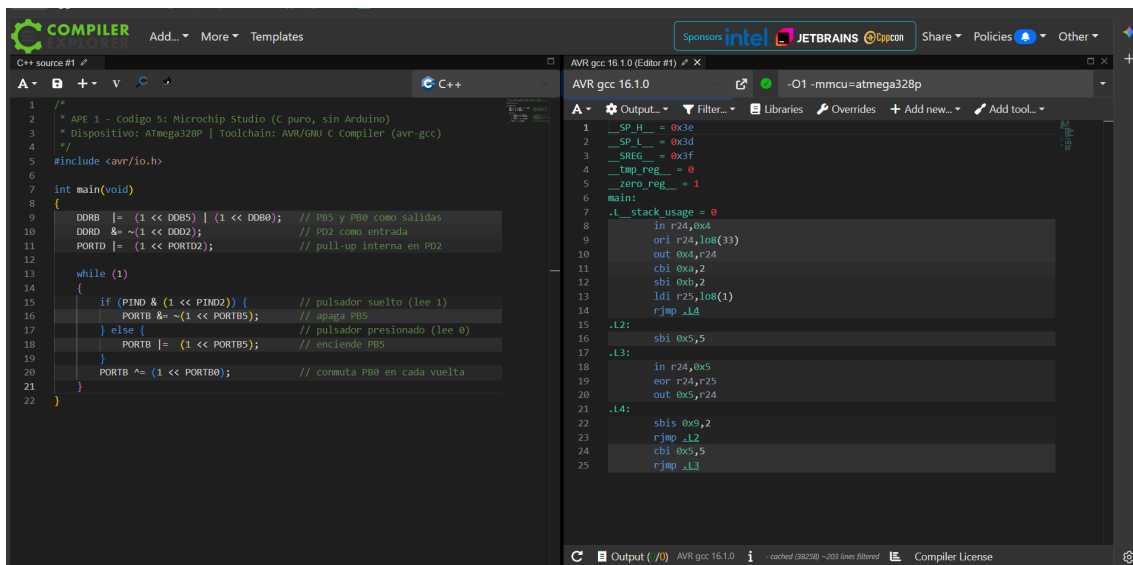


Tabla 4. Configuración de registros (para completar)

Pin	Función	Bit DDRx	Bit PORTx	Bit PINx leído
D8 (PB0)	LED0 encendido	1	0	—
D11 (PB3)	LED3 apagado	1	0	—
D2 (PD2)	Pulsador suelto	0	1	1
D2 (PD2)	Pulsador presionado	0	1	0
D13 (PB5)	LED "L" encendido	1	1	—

D2:

Tabla 5. Correspondencia C – ensamblador (para completar)



Línea en C	Instrucción(es) AVR	Ciclos
DDRB = (1 << DDB5) (1 << DDB0);	in r24,0x4 ori r24,lo8(33) out 0x4,r24	3 (1 por instrucción)
DDRD &= ~(1 << DDD2);	cbi 0xa,2	2
PORTD = (1 << PORTD2);	sbi 0xb,2	2
if (PIND & (1 << PIND2))	sbis 0x9,2	1 o 2
PORTB &= ~(1 << PORTB5);	cbi 0x5,5	2
PORTB = (1 << PORTB5);	sbi 0x5,5	2
PORTB ^= (1 << PORTB0);	in r24,0x eor r24,r25 out 0x5,r24	3 (1 por instrucción)

E2:

```
#include <avr/io.h>

/*
 * APE 1 - Código 5: Microchip Studio (C puro, sin Arduino)
 * @license: Atmega328P | Toolchain: AVR/GNU C Compiler (avr-gcc)
 */
#include <avr/io.h>

int main(void)
{
    DDRB |= (1 << DDB5) | (1 << DDB0); // PB5 y PB0 como salidas
    DDRD &= ~(1 << DDD2); // PD2 como entrada
    PORTD |= (1 << PORTD2); // pull-up interno en PD2

    while (1)
    {
        if (PIND & (1 << PIND2)) { // pulsador suelto (lee 1)
            PORTB &= ~(1 << PORTB5); // apaga PB5
        } else { // pulsador presionado (lee 0)
            PORTB |= (1 << PORTB5); // enciende PB5
        }
        PORTB ^= (1 << PORTB0); // conmuta PB0 en cada vuelta
    }
}
```

Output

```
Show output from: Build
Using "RunOutputFilterIfTask" task from assembly "C:\Program Files (x86)\Atmel\Studio7.0\Extensions\AvrGCC.d11".
Task "RunOutputFilterIfTask"
Program Memory Usage : 178 bytes 0.5 % Full
Data Memory Usage : 0 bytes 0.0 % Full
Warning: Memory Usage estimation may not be accurate if there are sections other than .text sections in ELF file
Done executing task "RunOutputFilterIfTask".
Done building target "CoreBuild" in project "GCCApplication1.proj".
Target "PostBuildEvent" skipped, due to false condition: ("$(PostBuildEvent)" != "") was evaluated as ("") != "".
Target "Build" in file "C:\Program Files (x86)\Atmel\Studio7.0\VsAvr\AvrCommon.targets" from project "C:\Users\miguelv\Documents\Atmel Studio7.0\Simulation\GCCApplication1\GCCApplication1.proj" (entry point):
Done building target "Build" in project "GCCApplication1.proj".
Done building project "GCCApplication1.proj".

Build succeeded.
----- Build: 1 succeeded or up-to-date, 0 failed, 0 skipped -----
```

El código se ejecutó y compiló sin ningún error

E3:

Tabla 5. Correspondencia C – ensamblador (para completar)

Línea en C	Instrucción(es) AVR	Ciclos
DDRB = (1 << DDB5) (1 << DDB0);	in, ori, out	3
DDRD &= ~(1 << DDD2);	in, andi, out	3
PORTD = (1 << PORTD2);	in, ori, out	3
if (PIND & (1 << PIND2))	sbis, rjmp	2
PORTB &= ~(1 << PORTB5);	in, andi, out, rjmp	4



PORTB = (1 << PORTB5);	in, ori, out	3
PORTB ^= (1 << PORTB0);	in, ldi, eor, out	4

The screenshot shows the Microsoft Dynamics 365 Development environment. The top menu bar includes File, Edit, View, View Tools, ASP, Project, Build, Debug, Tools, Window, Help. The top toolbar contains icons for various development actions. The main window displays a C# code file named 'Program.cs' with a line number 1. The code is a simple console application that prints 'Hello, World!' and exits. The debugger is set to 'Debug' mode, and the 'Breakpoints' pane is empty. The 'Registers' pane on the left shows the 'eax' register with a value of 0. The 'Memory' pane on the right shows a memory dump for 'prog TLASH' at address 0x40C000. The memory dump shows a series of hexadecimal values, mostly 00, with some non-zero values at the end of the dump.

The screenshot shows the Microchip Studio IDE with the following components:

- File Explorer:** Shows the project structure with files like `GccApplication1.c`, `GccApplication1.h`, and `main.c`.
- Code Editor:** Displays the C code for the AVR ATmega328P. The code is for a simple LED toggle program. The main function calls a while loop that toggles the state of PORTB pin 0 (PB0) every time a button connected to PORTD pin 2 (PD2) is pressed. The code includes comments in Spanish explaining the hardware and the logic.
- I/O View:** A window on the right side of the IDE showing the current state of the I/O components. It lists various I/O components like Analog Comparator (AC), Analog-to-Digital Converter (ADC), CPU Registers (CPU), EPROM (EEPROM), External Interrupts (EXTINT), I/O Port (PORTB), I/O Port (PORTC), I/O Port (PORTD), Serial Peripheral Interface (SPI), TimerCounter (Timer/Counter), and Watchdog Timer (WDT). The I/O Port (PORTB) component is selected, and its pins are listed with their current state (0 or 1).

The C code in the main.c file is as follows:

```
#include <avr/io.h>

/*
 * APE 1 - Código 5: Microchip Studio (C puro, sin Arduino)
 * Dispositivo: ATmega328P | Toolchain: AVR/GNU C Compiler (avr-gcc)
 */

#include <avr/io.h>

int main(void)
{
    DDHB |= (1 << DDHB5); // PB5 y PB0 como salidas
    DDDB |= (1 << DDDB5); // PD2 como entrada
    PORTD |= (1 << PORTD5); // pull-up interna en PD2

    while (1)
    {
        if (PIND & (1 << PIND5)) { // pulsador suelto (lee 1)
            PORTB ^= ~(1 << PORTB5); // toggle PB5
        } else { // pulsador presionado (lee 0)
            PORTB |= (1 << PORTB5); // enciende PB5
        }
        PORTB ^= (1 << PORTB5); // conmuta PB0 en cada vuelta
    }
}
```



The image displays two screenshots of the Microchip Studio IDE, showing the execution of an AVR program. The code is written in C and controls an LED (PES) and a buzzer (PBB) based on button presses (PIND2 and PORTB5).

Top Screenshot: The code is running, and the I/O View window shows the status of the I/O registers. The I/O View window is open, showing the I/O registers and their values. The I/O View window is open, showing the I/O registers and their values.

Name	Address	Value	Bits
PINB	0x23	0x20	00000000
DDRB	0x24	0x21	00000000
PORTB	0x25	0x20	00000000

Bottom Screenshot: The code is running, and the I/O View window shows the status of the I/O registers. The I/O View window is open, showing the I/O registers and their values.

Name	Address	Value	Bits
PINB	0x23	0x20	00000000
DDRB	0x24	0x21	00000000
PORTB	0x25	0x21	00000000

Un microprocesador como el de mi compu contiene solo la unidad de procesamiento (CPU) y necesita que le conectes memoria y periféricos externos para funcionar. Un microcontrolador como el ATmega328P es una "computadora en un solo chip": ya integra la CPU, la memoria (RAM y Flash) y los puertos de entrada/salida (I/O) en la misma pastilla.



- [illegible]

registros tomó solo 0.220 us por escritura. El acceso a registros es muchísimo más rápido porque escribe directamente en el hardware.

8. ¿Qué instrucción AVR generó `PORTB |= (1 << PORTB5);` y cuántos ciclos toma?

De acuerdo con la "Tabla 5" en el paso E3, generó las instrucciones `in`, `ori`, `out` y toma 3 ciclos. (Nota: En la Tabla 5 de la parte D2, también indicaste que se generó mediante `sbi 0x5`, 5 tomando 2 ciclos).

¿Por qué `PORTB ^= (1 << PORTB0);` no se tradujo a una sola instrucción?

Según la Tabla 5 del paso E3, esta línea en C se tradujo a las instrucciones AVR `in`, `ldi`, `eor`, `out`, tomando un total de 4 ciclos. No se hizo en una sola instrucción porque requiere varios pasos a nivel ensamblador para aplicar la operación lógica XOR (cargar el valor, realizar la operación exclusiva y sacarlo).

9. Sostenibilidad: indique dos formas en que lo trabajado (simular antes de comprar o montar, acceso eficiente a los periféricos) contribuye a reducir el consumo energético o los residuos electrónicos.

Simular en Wokwi antes de armar el circuito real evita que quemes componentes físicos (LEDs, resistencias o el propio microcontrolador) por cortocircuitos o malas conexiones, lo que significa menos placas arruinadas en la basura.

10. Ventajas y limitaciones de simular en Wokwi frente al hardware real (rebote del pulsador, límites de corriente, tiempos).

Es totalmente seguro e ideal para probar la lógica. No hay cables falsos que hagan mal contacto, no necesitas comprar hardware para aprender y puedes compartir el circuito con un enlace.

8. Conclusiones

- Durante esta práctica comprobé que manipular directamente los registros (DDR, PORT, PIN) del ATmega328P es drásticamente más rápido que usar la API nativa de Arduino. Medir los tiempos de ejecución me demostró que el acceso a registros toma apenas 0.220 microsegundos por escritura, frente a los 3.395 microsegundos de `digitalWrite`, un rendimiento que será clave cuando necesite programar aplicaciones donde el tiempo de respuesta sea crítico.
- El uso del simulador y de Microchip Studio me permitió entender lo que realmente ocurre dentro del microcontrolador a nivel de hardware. Observar cómo mis líneas de código en C se traducen a instrucciones específicas de ensamblador (como `sbi`, `cbi`, `in`, `out`), y ver cómo cambian los bits en la memoria paso a paso, me dio una perspectiva mucho más profunda sobre cómo optimizar mis programas en lugar de solo escribir código a ciegas.



9. Bibliografía

[1] W. P. Chamba Zaragocín, E. R. Guamán Quinche, F. J. Alvarez Pineda, J. O. Guamán Quinche, G. B. Zaragocín Arrobo y G. Á. Valarezo Benítez, Análisis y diseño preliminar de software basado en el modelado de requisitos y casos de uso. Un enfoque teórico y práctico. Loja, Ecuador: Universidad Nacional de Loja, 2025. Disponible en:
<https://unl.edu.ec/investigacion/produccion-cientifica/analisis-y-diseno-preliminar-de-software-basado-en-el-modelado>